



北京理工大學珠海學院

BEIJING INSTITUTE OF TECHNOLOGY, ZHUHAI

基于 Spark 分布式计算和 Hadoop 集群的银行客户忠诚度研究

学 院： 数理与土木工程学院

课 程： 大数据工具箱 2

班 级： 数据科学与大数据技术 2 班

姓 名： 肖宇涵 学 号： 211205101786

中国·珠海

2024 年 6 月 3 日

基于 Spark 分布式计算和 Hadoop 集群的银行客户忠诚度研究

摘 要

在本研究中，针对当前银行业产品同质化现象和客户忠诚度下降的问题，提出了一种基于客户数据分析的银行客户忠诚度提升方法。研究的核心在于通过构建银行客户忠诚度模型，以增强客户对银行产品和服务的依赖，进而提升银行的营销效果和客户忠诚度。

研究内容主要分为五个部分：首先是对客户数据进行预处理，确保数据的准确性和可用性；其次是基于客户的产品购买数据进行短期客户忠诚度分析，分析客户对银行产品的依赖程度；接着是利用客户资源信息进行长期客户忠诚度分析，旨在发现可能导致客户流失的因素；然后是基于长期客户数据分析构建客户忠诚度预测模型；最后是通过上述步骤完成客户忠诚度的综合分析，包括购买依赖度分析、客户流失因素分析及忠诚度预测的准确性。

技术上，本研究采用了 Spark 分布式集群和 Hive 数据仓库的技术方案。在 Linux 系统的虚拟机环境下，搭建了 Spark 分布式集群以高效利用计算资源，并通过 Hive 数据仓库对数据进行预处理，保证数据的准确性和一致性。此外，结合 Hadoop 和 Spark 进行数据存储与分析，实现了高效的企业财务数据分布式存储和处理。

总体上，本文共分为六章，从银行业务模式转变的必要性入手，通过技术实现对客户数据的有效分析，最终目的是通过科学的数据处理和分析提升银行的客户忠诚度和营销业绩。通过实验结果与分析，本研究验证了所提方法的有效性和实用性。

关键词：Spark Hadoop 银行客户忠诚度 数据分析

目 录

1	引言	1
2	文献综述	1
2.1	Spark 概述	1
2.1.1	Spark 的基本原理	1
2.1.2	Spark 的应用场景	1
2.2	数据仓库 Hive	2
2.2.1	Hive 的基本原理	2
2.2.2	Hive 在大数据处理中的作用	2
2.3	大数据处理与可视化	2
2.3.1	数据预处理与清洗	2
2.3.2	数据可视化技术	2
2.4	基于 PySpark 的模型构建	2
2.4.1	PySpark 简介	2
3	搭建 Spark 分布式集群	2
3.1	Spark 简介	2
3.2	Spark 安装	3
3.2.1	启动 Spark Shell	3
3.3	Spark 读取 HDFS 文件	4
4	搭建数据仓库 Hive	5
4.1	Hive 简介	5
4.2	Hive 与 Hadoop 生态系统中其他组件的关系	5
4.3	Hive	6
4.3.1	数据预处理	6
5	通过 Hadoop 集群进行数据可视化	7
5.1	相关系数矩阵图	7
5.2	不同年龄客户量占比的分组柱状图	8
5.3	蓝领与学生的产品购买情况饼图	8
5.4	以产品购买结果为 x 轴、拜访客户的通话时长为 y 轴的箱线图	9
5.5	绘制反映两种流失情况下不同年龄客户量占比的折线图	9
5.6	绘制反映两种流失情况下客户信用资格与年龄分布的散点图	10
5.7	绘制反映两种流失情况的客户各账号户龄占比量的堆叠柱状图	10

6 通过 PySpark 进行模型构建	11
6.1 数据预处理与特征选择	11
6.2 模型构建	11
6.3 随机森林 (Random Forest) 模型和算法简介	12
6.3.1 原理	12
6.3.2 构建过程	12
6.4 混淆矩阵 (Confusion Matrix)	13
6.4.1 定义	13
6.4.2 组成	13
6.4.3 主要指标	14
6.5 ROC 曲线 (Receiver Operating Characteristic Curve)	14
6.5.1 定义	14
6.5.2 关键概念	14
6.5.3 AUC (Area Under Curve)	14
6.6 训练模型与评估	14
6.7 结果与图表分析	16
7 讨论	18
7.1 模型表现分析	18
7.2 模型局限性与改进建议	18
8 总结与评价	19
8.1 总结	19
8.2 评价	19
参考文献	20
附录	21

1 引言

在当今的数字化时代，随着数据量的不断增长和数据类型的日益多样化，传统的数据处理框架已经难以满足大数据环境下的高效计算需求。**Apache Spark** 作为一种新兴的分布式计算框架，以其高速、灵活、易用的特点迅速获得了广泛的关注和应用。相较于传统的 **Map-Reduce** 框架，**Spark** 通过将数据处理和缓放入内存中，实现了更高的计算速度和效率，特别是在迭代计算和交互式数据分析方面展现出显著优势。

本研究旨在探讨如何在实际应用中构建和优化 **Spark** 分布式集群，以应对大数据环境下的复杂数据处理任务。首先，我们将介绍 **Spark** 的基本概念和工作机制，包括其核心组件和执行流程。随后，我们将展示如何将 **Spark** 与 **Hadoop** 结合使用，从而利用 **Hadoop** 分布式文件系统（**HDFS**）进行数据存储和访问。在此基础上，我们将详细描述 **Spark Shell** 的启动过程和通过 **Spark** 读取 **HDFS** 文件的方法。

随着数据量的激增，数据仓库工具的重要性日益凸显。本文还将介绍 **Hive** 的基本原理及其在 **Hadoop** 生态系统中的位置，探讨 **Hive** 与其他组件的关系以及 **Hive** 在数据预处理中的应用。通过对短期客户数据集的处理和分析，我们将展示如何使用 **Hive** 进行数据清洗、统计和分析，为后续的建模和预测提供基础数据支持。

在模型构建方面，我们将使用 **PySpark** 对银行客户的长期忠诚度进行预测，重点分析随机森林算法的原理和应用。针对数据不平衡、特征工程和超参数调优等问题，我们将提出相应的解决方案和改进建议。同时，通过构建和评估模型，我们将详细介绍混淆矩阵、**ROC** 曲线和 **AUC** 值等指标，深入分析模型的性能表现和应用效果。

本研究的最终目标是通过构建高效的 **Spark** 分布式集群和优化的数据处理流程，为大数据环境下的应用提供一套可行的解决方案。希望本研究能够为数据科学家和工程师在实际工作中有效利用 **Spark** 和 **Hive** 处理和分析大规模数据提供有价值的参考和指导。

2 文献综述

2.1 Spark 概述

2.1.1 Spark 的基本原理

Apache Spark 是一个快速、通用的大数据处理框架，其核心是通过在内存中进行数据处理，从而显著提高数据处理速度。与传统的 **MapReduce** 框架相比，**Spark** 在内存计算方面表现尤为突出，可以比 **MapReduce** 框架快上 100 倍 [4]。**Spark** 应用程序由一个驱动进程和多个执行进程组成，驱动进程负责任务调度和分发，执行进程则执行具体的计算任务并返回结果 [5]。

2.1.2 Spark 的应用场景

Spark 广泛应用于大数据处理、实时数据分析、机器学习等领域。在数据科学领域，**Spark** 因其内存计算能力和数据分布处理能力而受到青睐，尤其在优化大数据上的机器

学习算法方面表现突出 [6]。

2.2 数据仓库 Hive

2.2.1 Hive 的基本原理

Hive 是构建在 Hadoop 之上的数据仓库解决方案，提供了类似 SQL 的查询语言 HiveQL[7]。虽然 HiveQL 不完全遵循 SQL 标准，但其核心功能是将 HiveQL 语句转换为 MapReduce 任务，并通过批处理方式对大规模数据集进行分析 [8]。

2.2.2 Hive 在大数据处理中的作用

Hive 主要用于存储和查询 HDFS 上的静态数据，适合批处理大规模数据集。与 Pig 相比，Hive 更适用于结构化数据的处理，而 HBase 则提供实时数据访问能力，与 Hive 的静态数据处理形成互补 [9]。

2.3 大数据处理与可视化

2.3.1 数据预处理与清洗

大数据处理的第一步是数据预处理，包括数据清洗、缺失值处理和重复值处理。有效的数据预处理可以显著提高后续分析和模型构建的准确性 [10]。

2.3.2 数据可视化技术

数据可视化是数据分析的重要环节，通过图表形式直观展示数据特征和分析结果。本文中通过相关系数矩阵图、分组柱状图、饼图、箱线图和散点图等多种可视化手段，揭示了数据中的潜在模式和关系 [11]。

2.4 基于 PySpark 的模型构建

2.4.1 PySpark 简介

PySpark 是 Spark 的 Python API，以简洁易用的方式处理大规模数据。通过 PySpark，可以方便地进行数据预处理、特征选择和机器学习模型的构建 [12]。

3 搭建 Spark 分布式集群

3.1 Spark 简介

Apache Spark 是一个开源的分布式集群计算框架，用于快速处理、查询和分析大数据。它是当今企业中最有效的数据处理框架。通常依赖于 Map-Reduce 的框架的组织现

在正在转向 Apache Spark 框架。Spark 执行内存计算，比 Hadoop 等 Map Reduce 框架快 100 倍。Spark 在数据科学家中很受欢迎，因为它将数据分布和缓存放入了内存中，并且帮助他们优化大数据上的机器学习算法。

Spark 应用程序是 Spark 上下文的一个实例。它由一个驱动进程和一组执行程序进程组成。

驱动进程负责维护关于 Spark 应用程序的信息、响应代码、分发和调度执行器中的工作。驱动进程是非常重要的，它是 Spark 应用程序的核心，并在应用程序的生命周期内维护所有相关信息。

执行器负责实际执行驱动程序分配给他们的工作。因此，每个执行器只负责两件事：
执行由驱动程序分配给它的任务

将执行程序上的计算状态报告回驱动程序节点

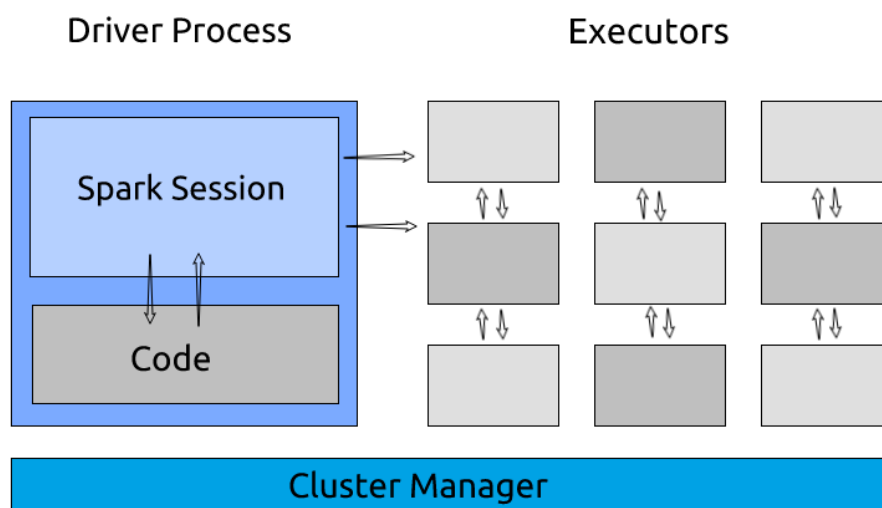


图 1: SPARK

3.2 Spark 安装

Spark 既可以独立安装使用，也可以与 Hadoop 结合使用。在本文中，我们与 Hadoop 一同安装，使 Spark 可以利用 HDFS 进行数据存储和访问。

3.2.1 启动 Spark Shell

Spark Shell 环境出现如下图图即为启动成功：

```

X _ □ 终端 文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
ss: 127.0.0.1; using 192.168.31.68 instead (on interface enp0s3)
23/12/26 23:40:37 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLeve
l(newLevel).
23/12/26 23:40:43 WARN NativeCodeLoader: Unable to load native-hadoop library fo
r your platform... using builtin-java classes where applicable
Spark context Web UI available at http://192.168.31.68:4040
Spark context available as 'sc' (master = local[*], app id = local-1703605244313
).
Spark session available as 'spark'.
Welcome to

      ____
     /___ \
    /   _ \
   /____ \|
  /_____ \
 /         \
/_          \
              version 3.5.0

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 1.8.0_292)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

图 2: sparkshell 模式

3.3 Spark 读取 HDFS 文件

随后，可以将银行客户忠诚度数据，包括长短期数据上传至 HDFS 的 `/user/hadoop` 目录下，使用 `hdfs dfs -put` 命令将文件上传到 HDFS，然后我们可以看出文件已经上传所在的 `/user/hadoop/` 目录了。

```

X  _  终端 文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
2477 NameNode
(base) hadoop@Master:/usr/local/hadoop$ hdfs dfs -put /home/hadoop/下载/short-customer-data.csv /user/hadoop/
(base) hadoop@Master:/usr/local/hadoop$ hdfs dfs -put /home/hadoop/下载/long-customer-train.csv /user/hadoop/
(base) hadoop@Master:/usr/local/hadoop$ hadoop fs -ls /
Found 3 items
drwxrwx--- - hadoop supergroup          0 2023-12-04 01:06 /tmp
drwxr-xr-x - hadoop supergroup          0 2023-12-04 15:35 /user
drwxr-xr-x - hadoop supergroup          0 2023-12-04 00:57 /usr
(base) hadoop@Master:/usr/local/hadoop$ hadoop fs -ls -h /user/hadoop/
Found 6 items
-rw-r--r-- 1 hadoop supergroup      6.9 M 2023-12-04 16:01 /user/hadoop/LR_new.csv
-rw-r--r-- 1 hadoop supergroup      1.7 M 2023-12-04 15:58 /user/hadoop/financial_data.csv
-rw-r--r-- 1 hadoop supergroup      916 2023-12-04 16:01 /user/hadoop/financial_data_new.csv
-rw-r--r-- 1 hadoop supergroup    418.0 K 2023-12-27 00:15 /user/hadoop/long-customer-train.csv
-rw-r--r-- 1 hadoop supergroup      3.5 M 2023-12-27 00:14 /user/hadoop/short-customer-data.csv
-rw-r--r-- 1 hadoop supergroup    204.2 K 2023-12-04 16:01 /user/hadoop/test.csv

```

图 3: HDFS 文件目录

4 搭建数据仓库 Hive

4.1 Hive 简介

Hive 是一个构建在 Hadoop 之上的数据仓库解决方案，由 Facebook 开发。它提供了类似 SQL 的查询语言—HiveQL，尽管不完全遵循 SQL 标准。例如，Hive 不支持更新操作、索引和事务，并且其子查询和连接操作有诸多限制。Hive 的核心功能是将 HQL 语句转换成 MapReduce 任务，并通过批处理方式对大规模数据集进行分析。作为一个数据仓库工具，Hive 非常适合存储和查询存放在 HDFS 上的静态数据。

4.2 Hive 与 Hadoop 生态系统中其他组件的关系

Hive 依赖于 HDFS 进行数据存储，并利用 MapReduce 进行数据处理。与 Hive 在 Hadoop 生态系统中的定位相比较，Pig 可被视为 Hive 的一个替代品，提供数据流语言和环境，主要用于查询半结构化数据集。而 HBase，作为一个面向列的分布式数据库，提供实时数据访问能力，与 Hive 的静态数据处理和 BI 报表生成功能形成互补。

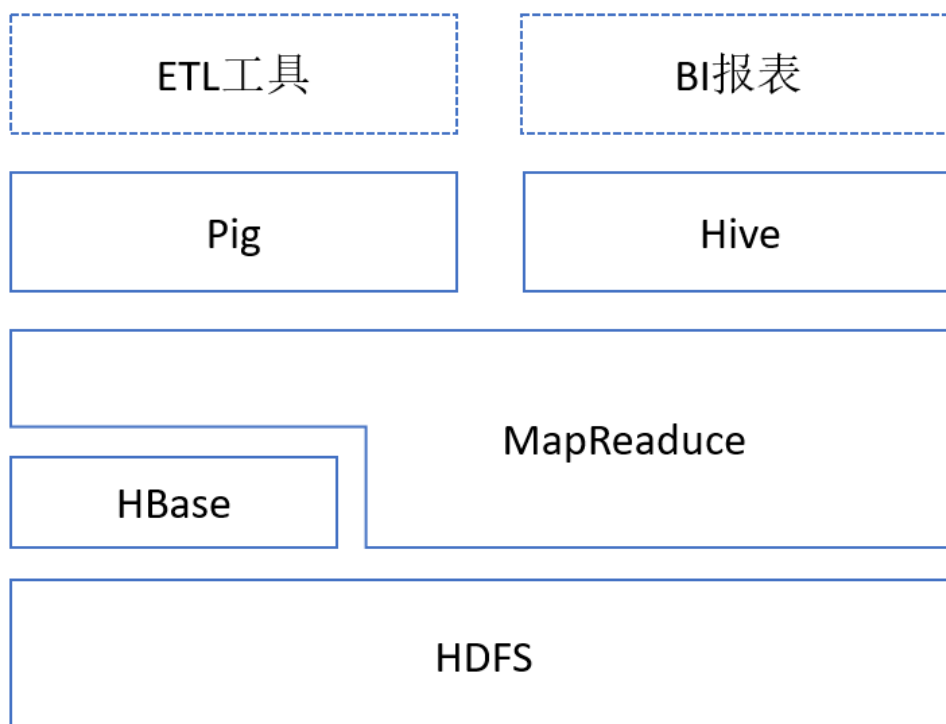


图 4: Hive 与 Hadoop 生态系统中其他组件的关系

4.3 Hive

4.3.1 数据预处理

由于 Hive 是建立在 Hadoop 之上的数据仓库工具，我们使用 HiveQL 语言编写的查询语句最终会被 Hive 自动转换成 MapReduce 任务，这些任务由 Hadoop 执行。因此，我们首先需要启动 Hadoop 系统，随后启动 Hive。成功启动的标志如下所示：

表 1: Hadoop 集群服务进程 ID

服务名称	进程 ID
NodeManager	3765
ResourceManager	3639
Jps	3800
DataNode	3261
NameNode	3134
SecondaryNameNode	3471

数据中存在部分缺失值，可能对后续任务产生影响，需要对数据进行清洗。为了使数据更加干净，包含缺失值的记录全部删去，为最大程度保留有效数据，缺失值处理在重复值处理之前进行。

短期客户产品购买数据集共有 41176 条记录，使用 DataFrame 的 count() 方法统计各列的非空数据量，统计结果如下表所示。

表 2: 数据列概览

#	数据列	非空数据量
1	user_id	41176
2	age	41176
3	job	40846
4	marital	41096
5	education	39446
6	default	32580
7	housing	40186
8	loan	40186
9	contact	41176
10	month	41176
11	day_of_week	41176
12	duration	41176

接下来，我们利用这些经过预处理的数据来计算不同企业的利润率和资产负债率，并将这些新计算出的数据插入到“利润率”和“资产负债率”字段中。同时，我们还会删除那些利润率和资产负债率不在合理范围（即不在 $[-300\%, 300\%]$ 范围内）的数据行，以优化表中的相关数据。

处理后，我们可以得到如下所示的前 5 个企业的利润率和资产负债率数据：

表 3: 企业利润率和资产负债率表

Stkcd	Accper	Typrep	利润率	资产负债率
600696	2018-03-31	A	46.150938	59.524425
547	2018-03-31	A	18.681072	22.588596
2772	2018-03-31	A	33.512902	34.062300
818	2018-03-31	A	19.188844	32.569762
300568	2018-03-31	A	62.315561	52.085464

5 通过 Hadoop 集群进行数据可视化

5.1 相关系数矩阵图

计算短期数据所有指标之间的相关性，绘制相关系数热力图。

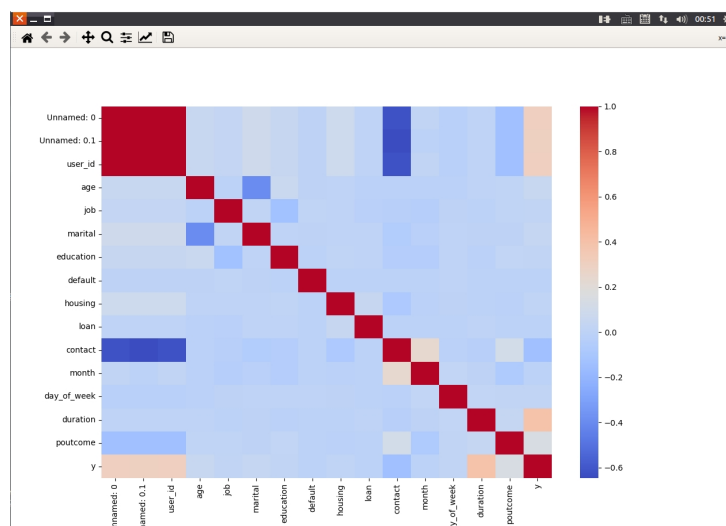


图 5: 相关系数矩阵图

由热力图可知，银行活动客户是否购买产品与最近一次拜访客户的通话时长、上一次银行活动呈现正相关关系，即随着通话时长的增加或上一次银行活动成功可能性的增加，银行客户更加愿意购买活动产品。银行活动客户是否购买产品与手机类型呈现负相关关系，即使用座机或传统手机的客户更加愿意购买产品。客户年龄与婚姻状况呈现负相关关系，即年龄越高，婚姻状态越容易趋向于离婚或丧偶。

5.2 不同年龄客户量占比的分组柱状图

绘制分组柱状图，展示了在两种产品购买结果（购买与未购买）下不同年龄客户的数量占比。X 轴代表客户的年龄，Y 轴代表各个年龄组中不同购买结果的客户比例。

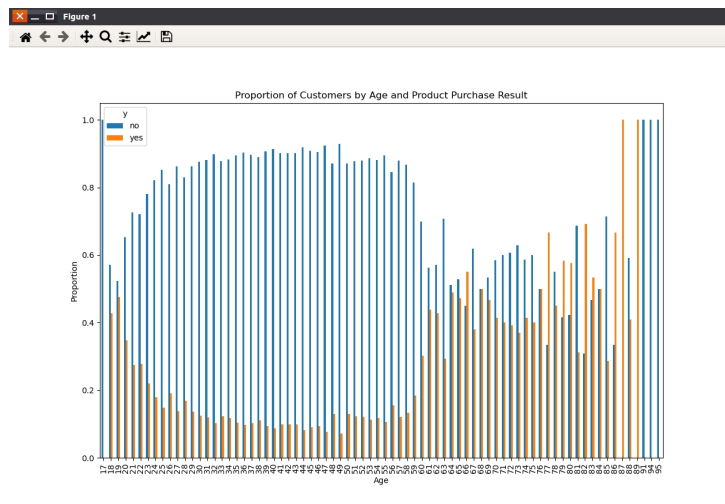


图 6: 不同年龄客户量占比的分组柱状图

5.3 蓝领与学生的产品购买情况饼图

绘制蓝领（blue-collar）与学生（student）的产品购买情况饼图，设定饼图的标签，显示产品购买情况的占比。

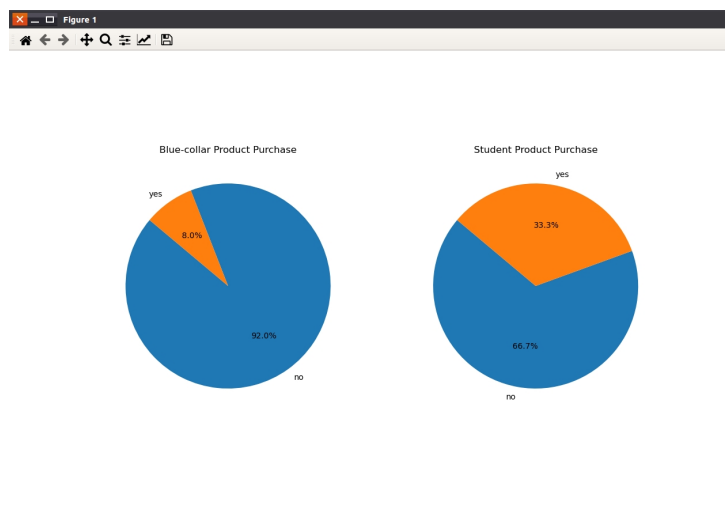


图 7: 蓝领与学生的产品购买情况饼图

从图中可知，职业为蓝领且未购买产品的人数占 83.1%，说明各个职业的未购买产品的人数占比可能均要大于购买产品的人数占比。

5.4 以产品购买结果为 x 轴、拜访客户的通话时长为 y 轴的箱线图

绘制箱线图，展示根据产品购买结果（x 轴）分类的拜访客户的通话时长（y 轴）。箱线图为我们提供了通话时长在不同购买结果（如购买或未购买）下的分布情况。

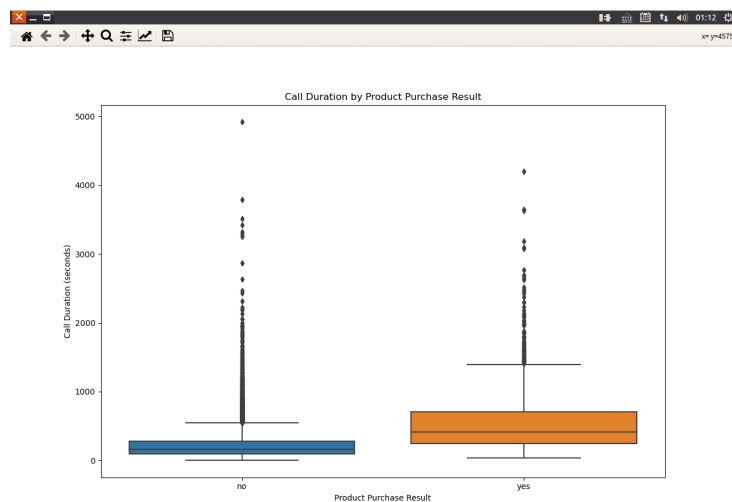


图 8: 以产品购买结果为 x 轴、拜访客户的通话时长为 y 轴的箱线图

任何一类购买情况的客户通话时长的中位数靠近下方，且中位数小于平均数，说明数据的分布呈现左偏分布。未购买产品客户的通话时长要比购买产品客户的通话时长更加集中，说明通话时长短是导致客户不购买产品的原因之一。

从箱体大小和位置分析，购买产品客户的箱体在于未购买产品客户的箱体之上，说明两种购买情况之间存在较大的差异。

5.5 绘制反映两种流失情况下不同年龄客户量占比的折线图

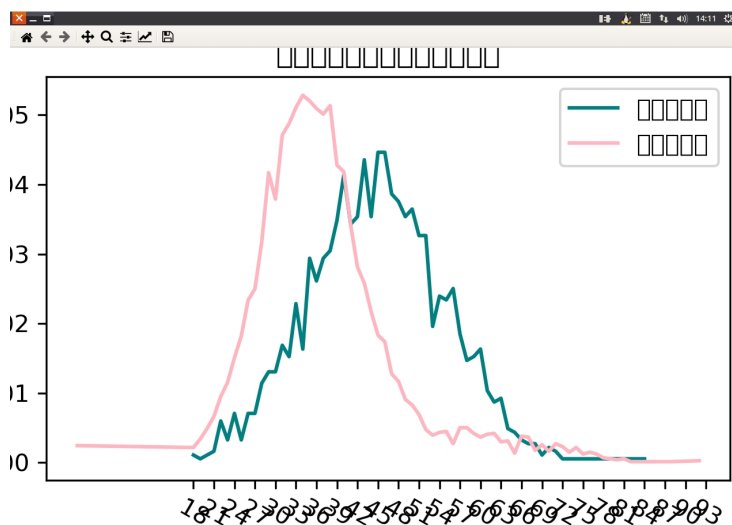


图 9: 绘制反映两种流失情况下不同年龄客户量占比的折线图

在长期数据的客户信息中，未流失的客户群体的年龄主要分布在 18 51 岁之间，已流失客户群体的年龄主要分布在 18 69 岁之间。

老年人群的流失情况较为严重，青壮年人群的保留情况较为好，说明长期数据中产品的受众主要为青壮年人群。

5.6 绘制反映两种流失情况下客户信用资格与年龄分布的散点图

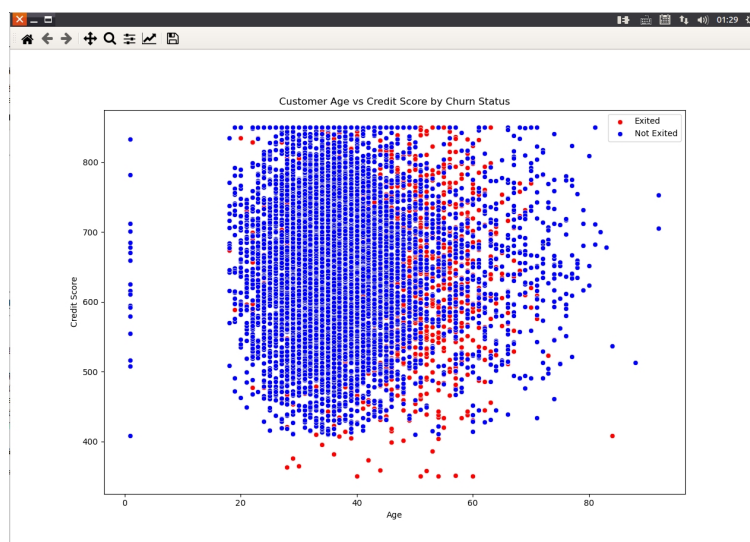


图 10: 绘制反映两种流失情况下客户信用资格与年龄分布的散点图

从上图可知，已流失客户的分布状况比较离散，未流失客户的分布状况整体较为离散，但在 20 50 岁之间存在较为集中的情况。

5.7 绘制反映两种流失情况的客户各账号户龄占比量的堆叠柱状图

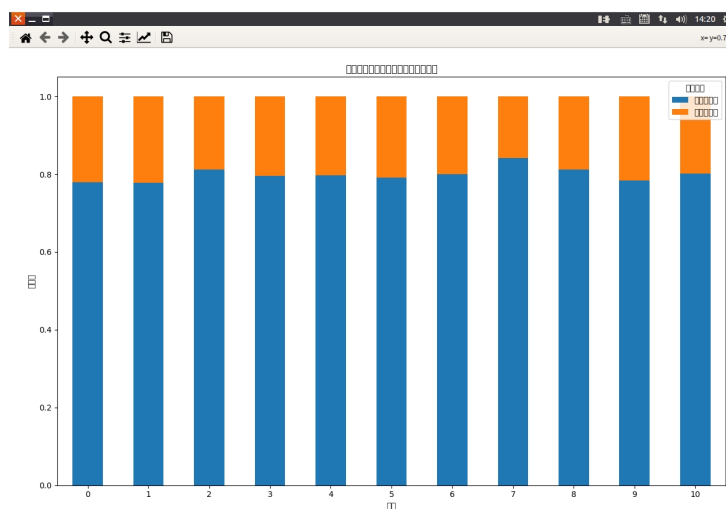


图 11: 包含各账号户龄在不同流失情况下的客户量占比透视表

通过上图分析可知，相同账户户龄的不同流失情况之间的关系较为稳定，并没有出现较大差异的账户户龄，说明不同账户户龄之间的流失情况不存在差异。同时，账户户龄为 0 和 11 年的人数占比较少，账户户龄的人数占比主要分布在 19 年之间。

6 通过 PySpark 进行模型构建

处理大数据的一种传统方式是使用像 Hadoop 这样的分布式框架，但这些框架需要在硬盘上执行大量的读写操作。事实上时间和速度都非常昂贵。计算能力同样是一个重要的障碍。

PySpark 以一种高效且易于理解的方式处理这一问题。由于上文我们已经安装与介绍过了 spark，故我们直接进行数据预处理与特征选择与模型的构建。长期数据存在“Exited”特征分布不均衡、各项数值分布跨度大等现象。体现为：未流失客户量是已流失客户量的 3 倍以上；客户信用资格最大数值达到 25 万，而客户活动状态则为 0 和 1 等。考虑上述现象，对银行客户长期忠诚度进行预测。

6.1 数据预处理与特征选择

在本研究中，我们聚焦于预测银行客户的流失情况。初始数据展示了特征“Exited”（客户是否流失）的分布不均，以及其他特征值的不同分布范围。例如，未流失的客户数量是已流失客户的三倍多；客户信用评分的最大值达到 25 万，而活动状态则是二分类变量（0 或 1）。针对这些特征，进行了适当的数据清洗和转换。

首先，我们移除了数据中的非关键列，如用户 ID 和其他无关标识符。其次，对于分类变量（如性别、是否持有信用卡等），采用了独热编码进行转换，以适应模型的需要。

6.2 模型构建

我们用 groupby 和 value_counts 看一下标签的分布状态。

表 4: 数据列概览

类别	样本量
0	7361
1	1837

从上表可知，数据之间存在不均衡的情况，所以模型构建时评价结果会倾向于大类样本的结果，导致模型的预测性较差。

不平衡数据的处理方法从大方向上分为两类——增加数据或改进算法。大多数情况下，很难增加具有真实意义的数据，因为数据的获取难度较大。所以增加数据是考虑对原始数据本身的特征和分布情况，通过一些方法进行填补补充，并不一定保证填补的数

据具有真实意义的效果。改进算法，一般采用的是对小类样本的计算过程更加偏向，通过不平等的分配权重改变预测结果。

6.3 随机森林（Random Forest）模型和算法简介

6.3.1 原理

随机森林的原理基于 Bagging 和随机化的思想。Bagging 是一种基于自助采样的集成学习算法，它通过从原始数据集中有放回地随机采样，生成多个数据集，并使用这些数据集训练多个分类器或回归器。随机化是一种随机选择特征的方法，它可以减少决策树的过拟合。

对于一个二分类或回归问题，假设我们的数据集是 $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ ，其中 x_i 是样本的特征向量， y_i 是样本的标签。我们使用 Bagging 的思想，从原始数据集中有放回地随机采样，生成 m 个数据集。对于每个数据集，我们使用随机化的方法，从原始特征中随机选择 k 个特征，构建一棵决策树。在构建决策树时，采用 CART 算法，选择最佳的特征进行分裂，并递归地构建子树。最终，我们得到了 m 棵决策树。

在进行分类或回归时，使用投票或平均的方法，将每个决策树的结果进行集成。对于分类问题，选择出现最多的类别作为最终的分类结果；对于回归问题，选择所有决策树的输出的平均值作为最终的回归结果。

随机森林的优点是能够处理高维数据和非线性可分问题，具有很好的泛化能力和较高的分类精度。同时，随机森林对于噪声数据比较鲁棒，不容易过拟合。但是随机森林也存在一些缺点，比如对于数据分布不平衡的问题，需要进行调整。

6.3.2 构建过程

1. 自助采样（Bootstrap Sampling）：从原始数据集中有放回地随机抽样，形成多个子集。
2. 构建决策树：对每个子集，随机选择特征子集，基于这些特征构建决策树。
3. 集成：
 - 分类任务：通过多数投票确定最终分类。
 - 回归任务：取多个决策树预测结果的平均值。



图 12: 随机森林的构建过程

6.4 混淆矩阵 (Confusion Matrix)

6.4.1 定义

混淆矩阵是用于评估分类模型性能的一种常见方法。混淆矩阵将真实标签和预测标签进行比较，从而得出模型的分类准确率、召回率、精度和 F1 分数等指标。混淆矩阵可以帮助我们了解分类模型的性能和误差类型，从而进一步优化模型。

混淆矩阵是一个二维矩阵，行表示真实标签，列表示预测标签。在二分类问题中，混淆矩阵通常由四个元素组成：真正例 (True Positive, TP)、假正例 (False Positive, FP)、真反例 (True Negative, TN) 和假反例 (False Negative, FN)。其中，真正例表示模型正确地将正例分类为正例的数量，假正例表示模型错误地将负例分类为正例的数量，真反例表示模型正确地将负例分类为负例的数量，假反例表示模型错误地将正例分类为负例的数量。

根据混淆矩阵的元素，我们可以计算出多个分类指标。准确率表示模型正确分类的样本数占总样本数的比例，计算公式为：

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \quad (1)$$

召回率 (Recall) 表示正例被正确分类的比例，计算公式为：

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

精度 (Precision) 表示被分类为正例的样本中真正为正例的比例，计算公式为：

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3)$$

F1 分数 (F1-Score) 是准确率和召回率的加权平均值，计算公式为：

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

6.4.2 组成

- **真正例 (True Positive, TP):** 正确地将正例预测为正例。
- **假正例 (False Positive, FP):** 错误地将负例预测为正例。
- **真负例 (True Negative, TN):** 正确地将负例预测为负例。
- **假负例 (False Negative, FN):** 错误地将正例预测为负例。

6.4.3 主要指标

- 准确率 (Accuracy): $\frac{TP+TN}{TP+FP+TN+FN}$
- 召回率 (Recall): $\frac{TP}{TP+FN}$
- 精确率 (Precision): $\frac{TP}{TP+FP}$
- F1 分数 (F1 Score): $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

6.5 ROC 曲线 (Receiver Operating Characteristic Curve)

6.5.1 定义

ROC 曲线是通过以真正例率为纵轴，假正例率为横轴，绘制而成的一条曲线，用于评估二分类模型的性能。真正例率表示模型将正例正确分类为正例的比例，计算公式为：

$$\text{TPR} = \frac{TP}{TP + FN} \quad (5)$$

假正例率表示模型错误地将负例分类为正例的比例，计算公式为：

$$\text{FPR} = \frac{FP}{FP + TN} \quad (6)$$

通过在不同阈值下计算模型在不同的真正例率和假正例率之间的权衡，可以绘制出 ROC 曲线。

在 ROC 曲线上，左下角是 (0,0) 点，表示将所有样本都预测为负例；右上角是 (1,1) 点，表示将所有样本都预测为正例。曲线离左上角越近，模型的性能越好。曲线下方的面积被称为 AUC (Area Under Curve)，AUC 越大，代表模型的性能越好。

6.5.2 关键概念

- 真正例率 (True Positive Rate, TPR): $\frac{TP}{TP+FN}$
- 假正例率 (False Positive Rate, FPR): $\frac{FP}{FP+TN}$

6.5.3 AUC (Area Under Curve)

AUC 衡量了 ROC 曲线下的面积，AUC 越大，模型性能通常越好。

6.6 训练模型与评估

在本研究中，我们选用随机森林算法来构建模型，其选择基于几个关键优势：随机森林能够高效处理大型数据集，并且在处理特征之间的非线性关系和相互作用方面表现出色。该算法通过构建多个决策树并汇总它们的预测结果，从而提高了预测的准确性，并有效控制了过拟合现象。

为了实现模型训练和评估，我们采用了 100 个决策树，并将数据集划分为 70% 的训练集和 30% 的测试集。这种划分确保了模型能在足够大的数据集上训练，同时保留了一部分数据用于评估模型的泛化能力。

进一步地，为了加强模型的构建和评估过程，我们引入了 PySpark 框架。首先，我们初始化了 Spark 会话，这为处理大规模数据提供了支持。在 PySpark 环境下，我们再次选择随机森林作为预测模型，以充分利用其处理大数据集和复杂特征关系的能力。在这个环境下，模型同样使用了 100 个决策树，并在划分好的训练集和测试集上进行了训练和评估。

通过这一系列的步骤，我们得到了模型的综合分析报告，该报告深入展示了模型在各方面的性能表现。

分类报告:

表 5: 分类性能指标

指标	分数 (%)
准确率 (0 类)	85
准确率 (1 类)	72
召回率 (0 类)	96
召回率 (1 类)	37
F1 分数 (0 类)	91
F1 分数 (1 类)	49
总体准确率	84

混淆矩阵:

表 6: 混淆矩阵结果

类别	数量
真正例 (0 类)	2108
假正例 (0 类)	83
假负例 (1 类)	358
真负例 (1 类)	211

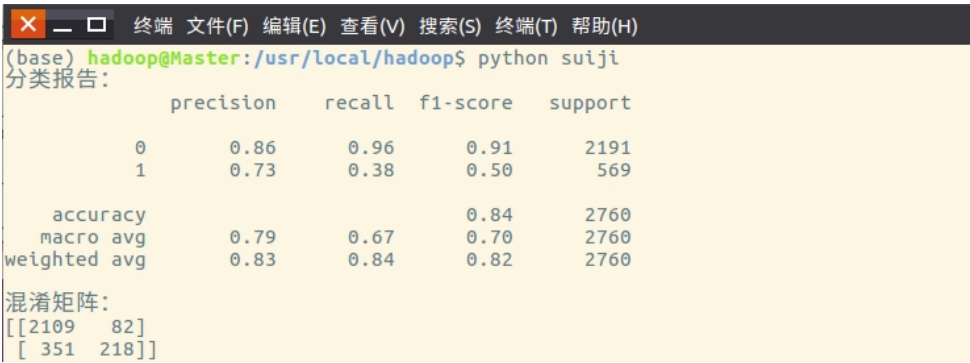


图 13: 随机森林报告

混淆矩阵的可视化如图 14。它展示了模型在预测不同类别时的性能。从图中可以看出，模型在识别未流失客户（类别 0）方面表现较好。然而，它在正确识别已流失客户（类别 1）方面存在一定的挑战，这可能是由于数据不平衡或其他潜在因素造成的。

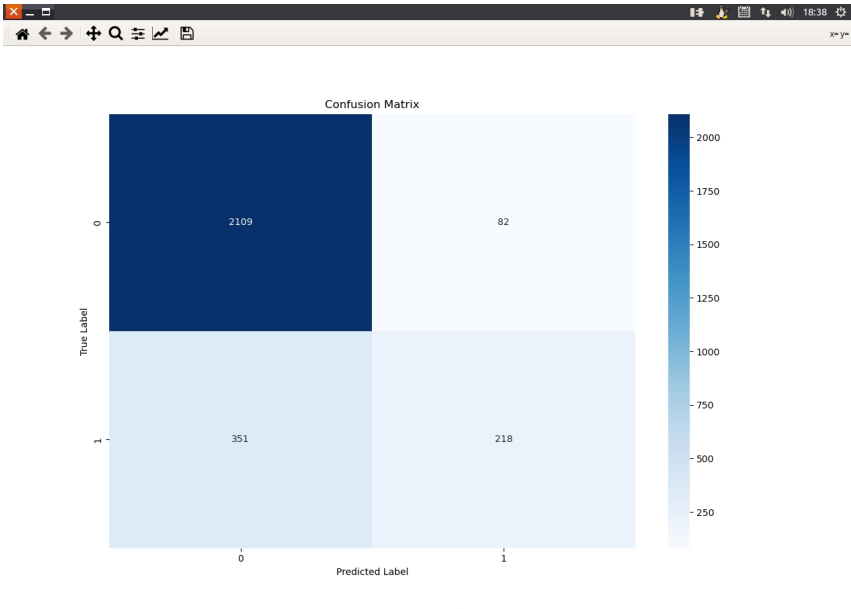


图 14: 混淆矩阵的热力图

6.7 结果与图表分析

特征重要性:

特征重要性图（见图 15）展示了各特征对模型预测结果的贡献程度。在我们的模型中，年龄、估计工资和信用评分是最重要的预测因素。这表明这些因素在银行客户流失问题上起着关键作用。

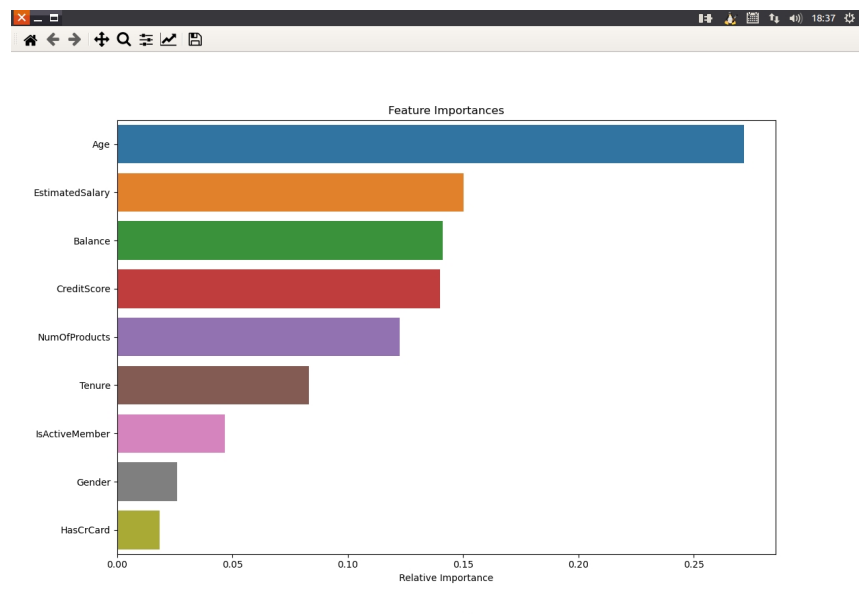


图 15: 特征重要性图

ROC 曲线与 AUC 值:

ROC 曲线及其 AUC 值（见图 ??）展示了模型在不同阈值下的表现。AUC 值反映了模型的整体分类能力。

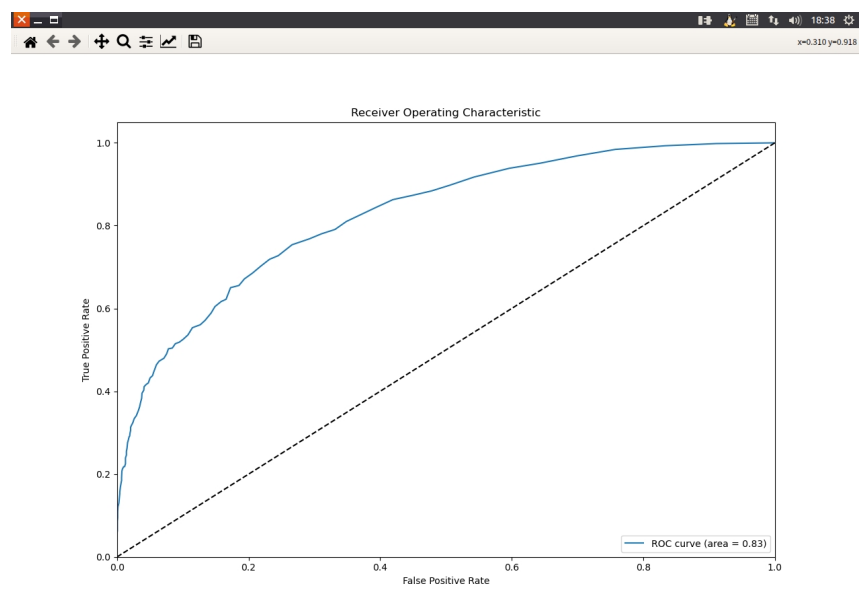


图 16: ROC 曲线

7 讨论

本研究通过应用随机森林模型预测银行客户流失，揭示了若干重要发现，并对这些发现的实践意义进行了深入探讨。模型的性能经过全面分析，特别是在正确识别流失客户（真正例）与误判非流失客户（假正例）的能力方面。

7.1 模型表现分析

根据特征重要性的分析，年龄、估计工资和信用评分是预测客户流失的关键因素。这一发现表明，随着年龄增长，客户流失的可能性增加，而较高的信用评分和工资水平似乎与较低的流失风险相关。这些洞察有助于银行更准确地识别影响客户忠诚度的因素，并据此制定有效的客户保留策略。

混淆矩阵结果揭示，尽管模型在识别未流失客户（类别 0）方面表现优异，但在准确识别已流失客户（类别 1）方面面临挑战。低召回率指出模型未能捕捉到一定比例的实际流失客户。此外，尽管模型整体精度较高，但存在一定比例的假正例，可能导致银行实施不必要的客户保留措施。

ROC 曲线和 AUC 值为模型整体性能提供了量化评估。虽然 AUC 值表明模型具备较好的区分能力，但在处理数据不平衡和避免过拟合方面仍有提升空间。

7.2 模型局限性与改进建议

数据不平衡：当前模型通过调整类别权重应对数据不平衡，但未来研究可考虑采用更先进的技术，如合成少数过采样技术（SMOTE），以提高对少数类别的预测准确性。

特征工程：模型性能可能受限于现有特征的范围。通过深入的特征工程，包括创建新特征或转换现有特征，可能揭示更多关于客户行为的信息，从而提升模型的预测精度。

模型复杂度：考虑到随机森林在处理复杂非线性模式方面的局限性，未来研究可能探索其他算法，如梯度提升机或神经网络，以实现更高效的模式识别。

超参数调优：细致的超参数调优可能进一步提升模型性能。采用网格搜索或随机搜索方法寻找最优模型参数，可能会提高模型的准确性和泛化能力。

8 总结与评价

8.1 总结

本文围绕 Hadoop 集群下银行客户忠诚度问题展开研究，采用数据科学和大数据技术进行深入分析。研究过程包括客户数据的预处理和特征工程，以及利用 Spark 分布式集群和 Hive 数据仓库进行高效数据处理和分析。通过搭建 Spark 分布式集群，本研究优化了计算资源的利用，并通过 Hive 数据仓库保障了数据的准确性和一致性。

研究通过可视化方法对客户忠诚度的多个方面进行了深入分析，包括相关系数矩阵、客户年龄分布、产品购买情况和客户流失情况等。这些可视化结果为后续的数据分析提供了宝贵的洞见。

在模型构建方面，本研究选用随机森林算法预测银行客户流失。该模型有效处理了数据不平衡问题，并在训练和测试集上进行了全面评估。分类报告和混淆矩阵的结果表明，模型在预测未流失客户方面表现良好，但在准确识别已流失客户方面存在一定挑战。

8.2 评价

技术应用的广度与深度：结合 Hadoop 集群、Spark 和 Hive 技术，本研究展现了在处理大规模银行数据方面的高效性。尤其在数据预处理和分析方面的应用，为银行客户忠诚度研究提供了新的视角。

数据分析与可视化：本研究在数据可视化方面表现出深度和细致，成功揭示了不同客户属性与流失情况的关系，有助于更好地理解影响客户忠诚度的关键因素。

模型选择与评估：采用随机森林算法基于其处理大规模数据集的能力。模型评估综合考量了准确率、召回率和 F1 分数，全面掌握了模型性能。

实用性与局限性：本研究提供了有效的数据处理方法和客户流失预测模型，但在处理已流失客户的预测方面显示出局限性，指出了未来研究的改进方向，如采用更先进的算法或进行深入的特征工程。

创新性：本研究在数据处理和分析方法上展现了创新，尤其是在集成多种大数据技术进行综合分析方面。同时，模型构建和评估也体现了对现有技术的深入应用和理解。

总而言之，本研究在银行客户忠诚度领域提供了新的思路和方法，特别是在数据处理和分析方面的深入探索，对于类似问题的研究具有重要的参考价值。然而，模型的改进和特征工程的深化仍然是未来研究的关键方向。

参考文献

- [1] 朱道恒. 基于 Hadoop 的语义 Web 服务发现的研究.
- [2] 胡健. 基于 Spark 的中文文本情感分析研究
- [3] 孙建华. Hadoop 和 Spark 技术在数据挖掘中的比较研究.
- [4] Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster Computing with Working Sets. Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing.
- [5] Karau, H., Konwinski, A., Wendell, P., & Zaharia, M. (2015). Learning Spark: Lightning-Fast Big Data Analysis. O'Reilly Media, Inc.
- [6] Armbrust, M., Das, T., Zaharia, M., Xin, R. S., Yavuz, B., & Stoica, I. (2015). Scaling Spark in the real world: Performance and usability. Proceedings of the VLDB Endowment.
- [7] Thusoo, A., Sarma, J. S., Jain, N., Shao, Z., Chakka, P., & Antony, S. (2009). Hive - A Petabyte Scale Data Warehouse Using Hadoop. 2009 IEEE 25th International Conference on Data Engineering.
- [8] Thusoo, A., Shao, Z., Sarma, J. S., Jain, N., Anthony, S., & Liu, H. (2010). Data warehousing and analytics infrastructure at Facebook. Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data.
- [9] Apache Hive Documentation. Retrieved from <https://cwiki.apache.org/confluence/display/Hive/Home>
- [10] García, S., Luengo, J., & Herrera, F. (2015). Data Preprocessing in Data Mining. Springer.
- [11] Few, S. (2012). Show Me the Numbers: Designing Tables and Graphs to Enlighten. Analytics Press.
- [12] Pentreath, N. (2015). Machine Learning with Spark. Packt Publishing Ltd.
- [13] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
- [14] Fawcett, T. (2006). An Introduction to ROC Analysis. Pattern Recognition Letters, 27(8), 861-874.
- [15] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357.

附录

附录 1：计算并显示相关系数矩阵的热力图

```

1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 from sklearn.preprocessing import LabelEncoder
5 from pyspark.sql import SparkSession
6
7 spark = SparkSession.builder.appName("HDFS_Read_Example").getOrCreate()
8 data = spark.read.csv("hdfs://namenode:8020/user/hadoop/long-customer-train.csv",
9                       header=True, inferSchema=True)
10
11 # 将分类变量转换为数值型
12 label_encoders = {}
13 for column in data.select_dtypes(include=['object', 'category']).columns:
14     label_encoders[column] = LabelEncoder()
15     data[column] = label_encoders[column].fit_transform(data[column].astype(str))
16
17 # 计算相关系数矩阵
18 correlation_matrix = data.corr()
19
20 # 设置绘图
21 plt.figure(figsize=(20, 18))
22
23 # 绘制热力图
24 sns.heatmap(correlation_matrix, cmap='coolwarm')
25
26 # 显示绘图
27 plt.show()

```

附录 2：绘制饼图，显示蓝领和学生的产品购买情况

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 spark = SparkSession.builder.appName("HDFS_Read_Example").getOrCreate()
5 data = spark.read.csv("hdfs://namenode:8020/user/hadoop/long-customer-train.csv",
6                       header=True, inferSchema=True)

```

```

7 # 筛选蓝领和学生的数据
8 blue_collar_data = data[data['job'] == 'blue-collar']
9 student_data = data[data['job'] == 'student']
10
11 # 统计产品购买情况
12 blue_collar_counts = blue_collar_data['y'].value_counts()
13 student_counts = student_data['y'].value_counts()
14
15 # 绘图
16 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7))
17
18 # 蓝领产品购买情况饼图
19 ax1.pie( blue_collar_counts , labels=blue_collar_counts.index, autopct='%1.1f%%',
20         startangle=140)
21 ax1.set_title('Blue-collar_Product_Purchase')
22
23 # 学生产品购买情况饼图
24 ax2.pie( student_counts , labels=student_counts.index, autopct='%1.1f%%', startangle
25         =140)
26 ax2.set_title('Student_Product_Purchase')
27
28 # 显示饼图
29 plt.show()

```

附录 3：制作分组柱状图，展示不同年龄客户的产品购买比例

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from pyspark.sql import SparkSession
4
5 spark = SparkSession.builder.appName("HDFS_Read_Example").getOrCreate()
6 data = spark.read.csv("hdfs://namenode:8020/user/hadoop/long-customer-train.csv",
7                       header=True, inferSchema=True)
8
9 # 按年龄和产品购买结果分组，并计算每组的客户数量
10 age_purchase_counts = data.groupby(['age', 'y']).size().unstack(fill_value=0)
11
12 # 计算每个年龄组内不同购买结果的客户比例
13 age_purchase_proportions = age_purchase_counts.div(age_purchase_counts.sum(axis=1),
14                                                     axis=0)

```

```

13
14 # 绘制分组柱状图
15 age_purchase_proportions.plot(kind='bar', figsize=(15, 8), stacked=False)
16
17 # 设置图表标题和坐标轴标签
18 plt.title('Proportion of Customers by Age and Product Purchase Result')
19 plt.xlabel('Age')
20 plt.ylabel('Proportion')
21
22 # 显示图表
23 plt.show()

```

附录 4：制作分组柱状图，展示不同年龄客户的产品购买比例

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from pyspark.sql import SparkSession
4
5 spark = SparkSession.builder.appName("HDFS_Read_Example").getOrCreate()
6 data = spark.read.csv("hdfs://namenode:8020/user/hadoop/long-customer-train.csv",
7                       header=True, inferSchema=True)
8
9 # 构建透视表，计算各户龄下的客户量占比
10 pivot = pd.pivot_table(df, values='CustomerId', index='Tenure', columns='Exited',
11                         aggfunc='count', fill_value=0)
12
13 # 计算占比
14 pivot_prop = pivot.div(pivot.sum(axis=1), axis=0)
15
16 # 绘制堆叠柱状图
17 pivot_prop.plot(kind='bar', stacked=True, figsize=(10, 6))
18
19 # 设置图表标题和轴标签
20 plt.title('客户各账号户龄占比量的堆叠柱状图')
21 plt.xlabel('户龄')
22 plt.ylabel('占比量')
23 plt.legend(['未流失客户', '已流失客户'], title='流失情况')
24 plt.xticks(rotation=0) # 户龄标签通常不需要旋转
25
26 # 显示图形

```

```

25 plt.tight_layout() # 用于确保所有标签都适合在图形中显示
26 plt.show()

```

附录 5：绘制饼图，显示蓝领和学生的产品购买情况

```

1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 from pyspark.sql import SparkSession
5
6 spark = SparkSession.builder.appName("HDFS_Read_Example").getOrCreate()
7 data = spark.read.csv("hdfs://namenode:8020/user/hadoop/long-customer-train.csv",
8                       header=True, inferSchema=True)
9
10 # 绘制箱线图
11 plt.figure(figsize=(10, 6))
12 sns.boxplot(x='y', y='duration', data=data)
13
14 # 设置图表标题和坐标轴标签
15 plt.title('Call_Duration_by_Product_Purchase_Result')
16 plt.xlabel('Product_Purchase_Result')
17 plt.ylabel('Call_Duration(seconds)')
18
19 # 显示图表
20 plt.show()

```

附录 6：制作分组柱状图，展示不同年龄客户的产品购买比例

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from pyspark.sql import SparkSession
5
6 spark = SparkSession.builder.appName("HDFS_Read_Example").getOrCreate()
7 data = spark.read.csv("hdfs://namenode:8020/user/hadoop/long-customer-train.csv",
8                       header=True, inferSchema=True)
9
10 # 绘制散点图
11 plt.figure(figsize=(12, 8))

```

```

12
13 # 绘制已流失客户的散点图
14 sns.scatterplot(x='Age', y='CreditScore', data=data[data['Exited'] == 1], color='red',
15               label='Exited')
16
17 # 绘制未流失客户的散点图
18 sns.scatterplot(x='Age', y='CreditScore', data=data[data['Exited'] == 0], color='blue',
19               label='Not_Exited')
20
21 # 设置图表标题和坐标轴标签
22 plt.title('Customer_Age_vs_Credit_Score_by_Churn_Status')
23 plt.xlabel('Age')
24 plt.ylabel('Credit_Score')
25
26 # 显示图例
27 plt.legend()
28
29 # 显示图表
30 plt.show()

```

附录 7：制作分组柱状图，展示不同年龄客户的产品购买比例

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 from pyspark.sql import SparkSession
5
6 spark = SparkSession.builder.appName("HDFS_Read_Example").getOrCreate()
7 data = spark.read.csv("hdfs://namenode:8020/user/hadoop/long-customer-train.csv",
8                       header=True, inferSchema=True)
9
10 # 创建已流失和未流失客户的数据框架
11 exited_yes = pd.DataFrame(data[data['Exited'] == 1]['Age'].value_counts().sort_index().reset_index())
12 exited_no = pd.DataFrame(data[data['Exited'] == 0]['Age'].value_counts().sort_index().reset_index())
13
14 # 转换年龄为整数型
15 exited_yes_x = exited_yes['index'].astype(int)

```

```

16 exited_no_x = exited_no['index'].astype(int)
17
18 # 计算占比
19 exited_yes_y = exited_yes['Age'] / exited_yes['Age'].sum()
20 exited_no_y = exited_no['Age'] / exited_no['Age'].sum()
21
22 # 绘制折线图
23 plt.figure(figsize=(15, 6), facecolor='#fff', dpi=300)
24 plt.plot(exited_yes_x, exited_yes_y, color='teal', label='已流失客户')
25 plt.plot(exited_no_x, exited_no_y, color='lightpink', label='未流失客户')
26
27 plt.xticks(range(18, 95, 3), rotation=330)
28 plt.xlabel('年龄')
29 plt.ylabel('占比')
30 plt.title('不同年龄客户的流失情况占比')
31 plt.legend()
32 plt.tight_layout()
33 plt.show()

```

附录 8：制作分组柱状图，展示不同年龄客户的产品购买比例

```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.metrics import classification_report, confusion_matrix
5 from pyspark.sql import SparkSession
6
7 spark = SparkSession.builder.appName("YourAppName").getOrCreate()
8 data = spark.read.csv("hdfs://namenode:8020/user/hadoop/long-customer-train.csv",
9                       header=True, inferSchema=True)
10
11 # 数据预处理
12 # 移除不必要的列
13 data_cleaned = data.drop(['Unnamed: 0', 'Unnamed: 0.1', 'CustomerId', 'Stausts', 'AsssceStage'], axis=1)
14
15 # 分离特征和目标变量
16 X = data_cleaned.drop('Exited', axis=1)
17 y = data_cleaned['Exited']

```

```
18
19 # 划分训练集和测试集
20 X_train, X_test, y_train, y_test = train_test_split (X, y, test_size =0.3,
    random_state=42)
21
22 # 随机森林模型（考虑类别不平衡）
23 clf_balanced = RandomForestClassifier( n_estimators=100, class_weight='balanced',
    random_state=42)
24 clf_balanced . fit (X_train, y_train )
25
26 # 预测和评估
27 y_pred_balanced = clf_balanced . predict (X_test)
28 classification_report_res  = classification_report ( y_test , y_pred_balanced)
29 confusion_matrix_res = confusion_matrix( y_test , y_pred_balanced)
30
31 # 打印评估结果
32 print ("分类报告： ")
33 print ( classification_report_res )
34 print ("混淆矩阵： ")
35 print ( confusion_matrix_res )
```